راهنمای جامع Query Monitor برای عیب‌یابی حرفه‌ای وردپرس

نسخه راهنما: ۱.۰ — ۲۶ اکتبر ۲۰۲۶

مخاطب: توسعه‌دهنده وردپرس با دانش PHP و SQL

مبنای محصول: Query Monitor 4.x؛ رابط و خروجی بعضی پنل‌ها بسته به نسخه، سایت و درخواست متفاوت است.

> **هدف این کتابچه:** بدون نیاز به حفظ‌کردن تمام منوها، از «علامت مشکل» به «شواهد»، «علت احتمالی»، «آزمایش اصلاحی» و «تأیید نتیجه» برسی. Query Monitor ابزار مشاهده و عیب‌یابی است، نه اثبات خودکار علت مشکل و نه جایگزین پروفایلر PHP، DevTools مرورگر یا APM.

فهرست راهبری سریع

1. روش استفاده از کتابچه و قواعد عیب‌یابی
2. معماری Query Monitor، نصب، مجوزها و `db.php`
3. نقشه کامل پنل‌ها و محدودیت‌ها
4. Overview و Timeline: تحلیل زمان و حافظه
5. Database Queries: کوئری‌ها، Caller، Component و Stack Trace
6. SQL پیشرفته: Slow / Duplicate / N+1 و EXPLAIN
7. Logs، PHP Errors، Doing it Wrong و Timings
8. HTTP API، REST API و AJAX
9. Conditionals، Template، Request و Rewrite
10. Hooks & Actions، Scripts، Styles و Block Editor
11. Capability، Cron، Transients، Object Cache و Environment
12. تمرین‌های ایمن PHP و JavaScript
13. هفت مسیر تصمیم‌گیری برای مشکلات واقعی
14. نمونه تحلیل Vinama
15. چک‌لیست‌های چاپی و قالب گزارش
16. واژه‌نامه، منابع و محدودیت‌ها

فصل ۱ — روش کار یک عیب‌یاب حرفه‌ای

چرخه استاندارد: مشاهده ← فرضیه ← آزمایش ← اندازه‌گیری

1. **بازتولید:** صفحه/endpoint، نقش کاربر، URL، پارامترها، محیط، زمان و وضعیت کش را ثبت کن.
2. **اندازه‌گیری اولیه:** Page Generation، Memory، زمان کوئری‌ها، HTTP calls و خطاها را یادداشت کن.
3. **تقسیم مسئله:** آیا گلوگاه دیتابیس است، تماس خارجی، کد PHP، کش، مرورگر، یا سرویس زیرساخت؟
4. **ردیابی:** Component و Caller را ببین؛ با Stack Trace به کد و خط فراخوانی برس.
5. **فرضیه قابل‌آزمایش:** «این حلقه برای هر محصول یک کوئری متادیتا ایجاد می‌کند» بهتر از «ووکامرس کند است» است.
6. **تغییر کم‌خطر و قابل‌برگشت:** تغییر در Staging، ثبت commit و تهیه نسخه پشتیبان پیش از اصلاح دیتابیس.
7. **تأیید:** چند اجرای قبل/بعد در شرایط مشابه، با کش سرد/گرم مشخص؛ مقایسه median و پراکندگی.
8. **مستندسازی:** علت تأییدشده، تغییر، نتیجه و اثر جانبی را ثبت کن.

هشدار: «یک Component بیشترین SQL Time را دارد» با «آن Component علت اصلی کندی سایت است» یکسان نیست. Query Monitor عمدتاً آنچه در همان درخواست PHP اتفاق افتاده را می‌بیند؛ اندازه‌گیری مرورگر و load-test را جدا انجام بده.

ترتیب پیشنهادی بررسی یک صفحه کند

`Overview → Timeline → Database Queries / HTTP API → Queries by Component / Caller → Stack Trace → آزمون هدفمند → اندازه‌گیری دوباره`

در آغاز، **URL دقیق، نقش کاربر، نوع درخواست (صفحه، Ajax یا REST)، کش فعال و ساعت آزمایش** را ثبت کن. نوسان روی لوکال می‌تواند از Xdebug، بار سیستم، آنتی‌ویروس و کش ناشی شود.

فصل ۲ — معماری، نصب و ایمنی

نصب و دسترسی

از مخزن رسمی وردپرس افزونه **Query Monitor** نوشته John Blackbourn را نصب کن. رابط در نوار مدیریت وردپرس ظاهر می‌شود؛ داده‌ها با توجه به سطح دسترسی نمایش داده می‌شوند. در سایت عمومی، نمایش جزئیات SQL، مسیر فایل و URLهای حساس را محدود نگه دار. «Authentication Cookie» فقط با آگاهی از خطرهای آن به کار رود.

Database Drop-in: چرا `db.php` مهم است؟

Query Monitor برای ثبت زودهنگام Stack Trace، SQL کامل، Component و تعداد ردیف‌ها از drop-in ویژه `wp-content/db.php` بهره می‌گیرد. هنگام فعال‌سازی تلاش می‌کند به فایل افزونه Symlink بسازد. **تنها یک `wp-content/db.php` می‌تواند وجود داشته باشد**؛ تعارض با drop-in افزونه‌های کش/دیتابیس محتمل است.

چک‌لیست خطای «Unable to symlink db.php»

- [ ] وجود یا نبود `wp-content/db.php` را بررسی کردم.
- [ ] اگر وجود دارد، قبل از هر اقدامی مشخص کردم متعلق به کدام افزونه است؛ کورکورانه حذف نکردم.
- [ ] مجوز نوشتن پوشه و مجوز Symlink را بررسی کردم.
- [ ] ابتدا راه استاندارد `wp qm enable` یا غیرفعال/فعال‌سازی را در محیط امن سنجیدم.
- [ ] در ویندوز فقط با مسیرهای درست و مجوز کافی از `mklink` استفاده کردم.
- [ ] بعد از رفع مشکل، ستون Component و Rows و بخش Queries by Component را دوباره دیدم.

مثال ویندوز مخصوص پروژه آموزشی Vinama (فقط در صورتی که هدف وجود دارد و مسیر همان است):

```
mklink "D:\www\vinama\wp-content\db.php" "D:\www\vinama\wp-content\plugins\query-monitor\wp-content\db.php"
```

برای دیدن مسیر Symlink در CMD: `dir /AL D:\www\vinama\wp-content`؛ قبل از تغییر یک فایل `db.php` موجود، محتوای آن و وابستگی سایر افزونه‌ها را مشخص کن.

تنظیمات و سربار

Query Monitor ابزار توسعه است و به‌ویژه در درخواست‌های سنگین، جمع‌آوری داده خودش سربار می‌آورد. برای بررسی دقیق، نتیجه را با/بدون ابزار نیز مقایسه کن. برای محیط تولید فقط با مدیریت مجوزها و سیاست امنیتی روشن استفاده کن.

فصل ۳ — اطلس پنل‌ها: کجا چه چیزی را ببینم؟

پنل	پاسخ به چه سؤالی؟	قدم بعدی
Overview	زمان، حافظه و خلاصه اجرای این درخواست چقدر است؟	Timeline
Timeline	کدام فعالیت‌ها و رویدادها در طول درخواست رخ داده‌اند؟	بخش مرتبط
Database Queries	SQL دقیق، زمان، نوع، Caller، Component و Rows چیست؟	Stack Trace / EXPLAIN
Duplicate Queries	آیا SQLهای یکسان تکرار شده‌اند؟	بررسی حلقه و cache
Queries by Caller	کدام توابع مجموع SQL بیشتری ایجاد کرده‌اند؟	Filter / Stack Trace
Queries by Component	کدام پلاگین/قالب/هسته زمان تجمعی SQL بیشتری دارد؟	SQL مربوط به همان بخش
PHP Errors	Warning/Notice/Deprecated یا خطاهای ثبت‌شده از کجا آمده‌اند؟	فایل و خط
Timings	تایمینگ‌های ثبت‌شده برای بخش‌های کد چه می‌گویند؟	مسیر کند
Logs	پیام‌های ثبت‌شده با ابزار لاگ QM چیست؟	context و مقادیر
HTTP API Calls	کدام درخواست‌های خروجی سرور کند/خطادار هستند؟	endpoint / timeout
Request	نوع و مشخصات درخواست چیست؟	Template/Route
Template	کدام فایل قالب انتخاب و بارگذاری شده؟	بررسی template hierarchy
Conditionals	شرط‌های وردپرس برای این درخواست چه نتیجه‌ای دارند؟	شرط اشتباه
Hooks & Actions	چه هوک‌هایی رخ داده‌اند و ترتیبشان چیست؟	callbackها
Scripts / Styles	چه assetهایی enqueue شده‌اند و وابستگی‌های آنها چیست؟	dependency/duplicate
Environment	نسخه PHP، وردپرس، سرور و تنظیمات مهم چیست؟	سازگاری و تنظیمات
Capability Checks	کدام بررسی مجوز برای کدام کاربر و کد صورت گرفته؟	نقش و capability
Transient Updates	چه transientهایی در درخواست ثبت/تغییر کرده‌اند؟	کش و عملیات نوشتنی
Languages / Block Editor / سایر پنل‌ها	مشکلات ترجمه، بلوک و فناوری‌های فعال؟	مسیر وابسته به پیکربندی

مهم: وجود یا نام برخی پنل‌ها به نوع درخواست و افزونه‌های فعال بستگی دارد. Query Monitor 4 رابط Preact و Timeline جدید دارد. دکمه‌ها و رفتار مرتب‌سازی را در نسخه فعلی بررسی کن؛ روی کلیک‌پذیر بودن یک تیتر بدون نشانه رابط حساب نکن.

فصل ۴ — Overview و Timeline

خواندن Overview

- **Page Generation Time:** زمان تولید پاسخ توسط سمت سرور در چارچوب اندازه‌گیری افزونه؛ معادل LCP یا زمان بارگذاری کامل مرورگر نیست.
- **Peak Memory Usage:** اوج استفاده PHP از حافظه در درخواست؛ با `memory_limit` و نوع صفحه مقایسه کن.
- **Database Query Time:** مجموع زمان SQL ثبت‌شده؛ لزوماً کل زمان انتظار پایگاه داده یا تمام هزینه لایه persistence نیست.
- **Database Query Count:** تعداد کوئری؛ صرف زیادبودن نشانه باگ نیست.

مثال تشخیص

اگر تولید پاسخ ۲٫۵ ثانیه و SQL تنها ۰٫۰۵ ثانیه است، مستقیم سراغ index گذاری نرو؛ HTTP API، Timeline و تایمینگ PHP را بررسی کن. زمان بخش‌ها ممکن است روی یکدیگر هم‌پوشانی داشته باشند؛ تفریق ساده، «زمان دقیق PHP» را نمی‌دهد.

Timeline چگونه به کار می‌رود؟

1. درخواست را دو تا سه بار بازتولید کن.
2. Timeline را باز کن و بخش‌های طولانی/انتظارها را تشخیص بده.
3. رویداد را به پنل مربوطه ربط بده: PHP errors، Timings، HTTP، DB یا hooks.
4. اگر زمان با داده‌های QM توضیح داده نمی‌شود، سراغ پروفایلر PHP مانند Xdebug profiler/Blackfire یا اندازه‌گیری اختصاصی برو.

قاعده: Timeline مسیر تحقیقات را پیشنهاد می‌کند، نه اینکه به‌تنهایی علت قطعی باشد.

فصل ۵ — Database Queries: از SQL تا کد PHP

ستون‌ها

- **Query:** متن SQL اجراشده.
- **Caller:** تابع مرتبط با ایجاد/اجرای SQL (ممکن است تابعی در هسته باشد).
- **Component:** نسبت‌دادن کوئری به افزونه، قالب یا هسته؛ با مالک جدول یکی نیست.
- **Rows:** تعداد ردیف‌های بازگشتی/متأثر، با توجه به نوع SQL و شیوه ثبت.
- **Time:** زمان اجرای همان SQL، معمولاً در واحد ثانیه در رابط.
- **Stack Trace:** زنجیره فراخوانی‌ها؛ برای پیدا کردن نقطه‌ای از کد پروژه که می‌توان اصلاح کرد.

روش صحیح خواندن Stack Trace

در نمونه واقعی Vinama:

```
front-page.php
  -> get_footer()
    -> footer.php
      -> wp_nav_menu()
        -> wp_get_nav_menu_items()
          -> get_posts()
            -> WP_Query->query()
              -> WP_Query->get_posts()
                -> SQL
```

این مسیر نشان می‌دهد فایل `front-page.php` فوتر را بارگذاری کرده و دریافت آیتم‌های منو در زنجیره اجرای فوتر رخ داده است. برای اثبات دقیق محل فراخوانی، **نام فایل و شماره خط فریم مربوطه** را باز کن؛ حضور نام یک فایل در Stack Trace به‌تنهایی اثبات باگ همان فایل نیست.

By Caller در برابر By Component

- **By Caller:** جمع زمان/تعداد SQL بر اساس تابع؛ مثلاً `update_meta_cache` .
- **By Component:** جمع زمان/تعداد SQL بر اساس بخش نسبت‌داده‌شده؛ مثلاً Rank Math یا Theme.
- یک Caller می‌تواند در چند Component ظاهر شود؛ از این نتیجه نگیر تابع چند بار تعریف شده است.
- زمان SQL یک افزونه برابر با کل هزینه آن افزونه نیست.

فرآیند تشخیص کوئری مشکوک

1. فیلتر نوع SQL را تعیین کن: SELECT یا Non-SELECT.
2. زمان **منفرد** و **تجمعی** را از هم تفکیک کن.
3. همان SQL را بررسی کن: WHERE، JOIN، ORDER، LIMIT و ستون‌های انتخاب‌شده.
4. Caller، Component و Stack Trace را ثبت کن.
5. بررسی کن کوئری یک بار اجرا می‌شود یا در حلقه تکرار دارد.
6. فقط اگر شواهد لازم بود، EXPLAIN و تغییرات قابل‌برگشت انجام بده.

فصل ۶ — N+1، Duplicate، Slow Queries و EXPLAIN

Slow و Duplicate دو چیز متفاوت‌اند

- **Slow Query:** یک اجرای SQL به شکل غیرمنتظره زمان‌بر است.
- **Duplicate Query:** SQL یکسان بیش از یک بار اجرا شده؛ ممکن است طبیعی باشد یا نشانگر فراخوانی تکراری.
- **N+1:** بعد از دریافت N شیء، برای هر شیء کوئری مرتبط جداگانه اجرا می‌شود. ممکن است کوئری‌ها یکسانِ متنی نباشند؛ Duplicate panel همه انواع N+1 را شکار نمی‌کند.

مثال N+1 (نمای آموزشی)

```
$posts = get_posts(['post_type' => 'post', 'numberposts' => 20]);
foreach ($posts as $post) {
    // اگر prime شدن متادیتا غیرفعال شده باشد یا الگو مناسب نباشد،
    // دریافت داده مرتبط ممکن است باعث درخواست‌های مجزا شود.
    $value = get_post_meta($post->ID, 'custom_key', true);
}
```

هشدار فنی: `get_posts` در شرایط معمول می‌تواند cache متادیتا را از قبل prime کند؛ این مثال اثبات N+1 در همه اجراها نیست. ابتدا در Query Monitor ببین آیا واقعاً SQLهای جداگانه ایجاد شده‌اند.

اجرای EXPLAIN

برای SELECT مشکوک، در محیط امن و با نام جدول صحیح:

```
EXPLAIN SELECT post_id
FROM wp_postmeta
WHERE meta_key = '_custom_flag'
  AND meta_value = 'active';
```

معنای ستون‌های رایج:

ستون	تفسیر
`type`	نوع روش دسترسی به جدول؛ ALL معمولاً یعنی full scan
`possible_keys`	ایندکس‌هایی که ممکن است به کار بیایند
`key`	ایندکس واقعاً انتخاب‌شده (اگر وجود دارد)
`rows`	تعداد تخمینی ردیف‌های موردبررسی، نه نتیجه قطعی
`filtered`	درصد تخمینی ردیف‌هایی که شرط فیلتر را می‌گذرانند
`Extra`	سرنخ‌هایی مانند Using where، Using temporary و Using filesort

نکته: `Using filesort` همیشه مشکل نیست؛ `type=ALL` روی جدول کوچک هم ممکن است بهترین برنامه باشد. EXPLAIN تخمینی است؛ در MySQL مناسب می‌توان از EXPLAIN ANALYZE استفاده کرد اما آن دستور واقعاً query را اجرا می‌کند؛ هرگز بی‌برنامه روی دستورهای تغییردهنده یا داده حساس اجرا نکن.

چک‌لیست اصلاح SQL

- ☐ آیا `SELECT *` واقعاً لازم است؟
- ☐ شرط WHERE انتخاب‌پذیر و متناسب با داده است؟
- ☐ JOIN روی ستون‌های درست است؟
- ☐ ORDER و GROUP واقعاً لازم‌اند؟
- ☐ آیا index موجود استفاده می‌شود؟
- ☐ آیا مشکل از تعداد دفعات اجراست، نه سرعت یک اجرا؟
- ☐ آیا Object Cache/priming مناسب‌تر از تغییر SQL است؟
- ☐ تست قبل/بعد در حجم داده شبیه تولید انجام شد؟

هشدار: ایجاد ایندکس روی جداول وردپرس هزینه نوشتن و مصرف دیسک دارد؛ روی `meta_value` با نوع LONGTEXT، طراحی ایندکس مستقیم محدودیت و tradeoff دارد. تغییر schema فقط بعد از بررسی واقعی workload.

فصل ۷ — PHP Errors، Logs و Timings

Error panels را چگونه بخوانی؟

1. **Severity:** Notice، Warning، Deprecated و دیگر رویدادهای گزارش‌شده را از هم جدا کن.
2. **Message:** متن خطا؛ تابع/آرگومان/نوع داده را استخراج کن.
3. **File/Line:** محل وقوع خطا (ممکن است داخل هسته باشد).
4. **Stack Trace:** کدی را که API را اشتباه صدا زده پیدا کن.
5. **Reproduction:** آیا فقط در PHP جدید، درخواست خاص یا role خاص اتفاق می‌افتد؟

مثال: هشدار ترجمه زودهنگام `_load_textdomain_just_in_time` اگر از `wp-includes/functions.php` گزارش شود لزوماً مشکل از فایل هسته نیست؛ Stack Trace را دنبال کن تا ببینی چه قالب یا افزونه‌ای ترجمه را زودتر از هوک مناسب فراخوانده است.

Logs

وقتی افزونه از API لاگ Query Monitor پشتیبانی می‌کند، پیام‌ها در پنل Logs نمایان می‌شوند. در کد اختصاصی می‌توان از توابع `do_action('qm/debug', ...)` و سطوح مشابه در چارچوب مستندات افزونه بهره برد؛ قبل از استفاده نسخه و API دقیق را بررسی کن. هیچ رمز، token یا داده شناسایی کاربر را لاگ نکن.

نمونه احتیاط‌آمیز در لوکال:

```
if (defined('WP_DEBUG') && WP_DEBUG) {
    do_action('qm/debug', 'Vinama: starting product query');
}
```

Timings و پروفایلینگ

برای اندازه‌گیری بخش‌های سفارشی، از Timings در نسخه و افزونه مناسب استفاده کن یا با `microtime(true)` بازه را اندازه بگیر. اندازه‌گیری `microtime` زمان wall-clock یک قطعه را نشان می‌دهد و جایگزین profiler CPU نیست.

```
$start = microtime(true);
// کاری که می‌خواهی اندازه بگیری
$elapsed_ms = (microtime(true) - $start) * 1000;
error_log(sprintf('Operation: %.2f ms', $elapsed_ms));
```

لاگ نمونه را فقط در محیط توسعه و بدون داده حساس فعال کن.

فصل ۸ — HTTP API، REST و AJAX

HTTP API Calls

بررسی کن:

- مقصد درخواست و کدام کد آن را ساخته؟
- کد وضعیت (۲xx، ۴xx، ۵xx)، خطای DNS/TLS/timeout و مدت پاسخ چیست؟
- آیا تماس خارجی در مسیر رندر صفحه و به شکل synchronous اجرا می‌شود؟
- آیا timeout، caching یا queue/background job مناسب است؟

نمونه: صفحه در ۲ ثانیه تولید می‌شود، SQL تنها ۴۰ms و یک API بیرونی ۱٫۶s طول می‌کشد. اول endpoint و Stack Trace تماس را بررسی کن، نه index های دیتابیس.

REST و AJAX

این درخواست‌ها ممکن است جدا از بارگذاری اولیه صفحه باشند. برای خطای یک endpoint، درخواست هدف را در Network مرورگر شناسایی کن؛ سپس داده Query Monitor مربوط به **همان درخواست** را بررسی کن. داده درخواست صفحه اصلی، به‌طور خودکار SQLهای فراخوانی AJAX بعدی را شامل نمی‌شود.

فصل ۹ — Conditionals، Template، Request و Rewrite

این پنل‌ها بیشتر برای «چرا این صفحه محتوای اشتباه دارد؟» مفیدند:

- **Request:** نوع درخواست و متغیرهای کوئری را بررسی کن.
- **Template:** ببین چه قالبی واقعاً انتخاب شده است؛ `front-page.php` ، `page.php` ، قالب بلوکی یا فایل سفارشی؟
- **Conditionals:** نتیجه `is_front_page()` ، `is_home()` ، `is_single()` و شرط‌های مرتبط را بررسی کن؛ `is_front_page()` و `is_home()` همیشه یکسان نیستند.
- **Rewrite:** برای 404های غیرمنتظره، ساختار rewrite و route را با URL مقایسه کن؛ در صورت نیاز rewrite rules را به‌شکل کنترل‌شده بازسازی کن.

سناریو: صفحه آرشیو قالب اشتباه دارد → Request / query vars → Template → Conditionals → قوانین rewrite؛ قبل از دست‌کاری permalink، URL و post type را ثبت کن.

فصل ۱۰ — Hooks، Scripts، Styles و Block Editor

Hooks & Actions

از پنل برای مشاهده hookهای اجراشده و شناسایی مسیر زمانی استفاده کن. وقوع یک hook به‌تنهایی ثابت نمی‌کند callback سنگینی دارد؛ برای اندازه‌گیری اجرای callback از ابزار اندازه‌گیری یا پروفایلر استفاده کن.

نمونه ردیابی

```
add_action('wp_footer', function () {
    if (defined('WP_DEBUG') && WP_DEBUG) {
        do_action('qm/debug', 'wp_footer reached');
    }
}, 999);
```

Scripts / Styles

بررسی کن handle، URL، نسخه، dependency، parent/dependent، محل و ترتیب enqueue چیست. مشکل‌هایی مانند missing dependency یا بارگذاری دو نسخه library را پیدا کن. Query Monitor سرور-side assets را نشان می‌دهد؛ برای blocking، transfer size و اجرای JS به DevTools → Network / Performance هم نیاز است.

Block Editor

پنل‌های مربوط به بلوک‌ها فقط در درخواست‌ها و پیکربندی‌های مناسب ظاهر می‌شوند. وقتی رندر بلوک سروری کند است، با Timing و Stack Trace رندر callback آن را بررسی کن.

فصل ۱۱ — Cache، Transients، Cron، Capability و Environment

Object Cache و Transients

- Object Cache ذخیره موقت داده‌ها برای کم‌کردن کوئری‌های تکراری است؛ persistent بودن آن وابسته به راه‌اندازی Redis/Memcached و drop-in مربوط است.
- Transient با TTL برای ذخیره موقت داده است؛ ثبت/تمدید مکرر آن ممکن است عملیات UPDATE ایجاد کند.
- `update_option` یا `set_transient` در هر بار بازدید الزاماً bug نیست، ولی اگر خروجی آن ثابت است، بررسی کن چرا هر بار نوشته می‌شود.
- توجه کن که **Component = Core** برای `update_option` می‌تواند همراه با نام transient یک افزونه باشد؛ برای یافتن منبع مستقیم به Stack Trace نیاز است.

Cron و زمان‌بندی

رویدادهای WP-Cron ممکن است هنگام بازدید فعال شوند. اگر کندی تناوبی است، زمان وقوع را با رخدادهای Cron و درخواست‌های HTTP مقایسه کن. اجرای خودکار cron و endpoint آن را با روش مناسب و ابزارهای دیگر نیز بررسی کن؛ Query Monitor به‌تنهایی dashboard کامل زمان‌بندی نیست.

Capability Checks

اگر مدیری به بخشی دسترسی ندارد، بررسی کن `current_user_can` یا متدهای مرتبط در چه جایی و با چه capability صدا شده‌اند. Capability و role را یکی ندان؛ کنترل مجوز وابسته به نقش، meta capabilities و filterهاست. روی داده حساس در خروجی این پنل احتیاط کن.

Environment

نسخه PHP/MySQL/MariaDB، حافظه مجاز، کش و پیکربندی مرتبط را ثبت کن. ناسازگاری افزونه با نسخه PHP می‌تواند warning/notice یا رفتار نادرست ایجاد کند.

فصل ۱۲ — ابزارهای آماده و تمرین‌های ایمن

۱۲.۱ نمایش ۱۰ کوئری کندتر از QueryMonitorData (نسخه ۴)

در مرورگر، با DevTools → Console و فقط برای همان درخواست:

```
(() => {
  const rows = QueryMonitorData?.data?.db_queries?.data?.rows;
  if (!Array.isArray(rows)) {
    console.warn('QM query rows not available');
    return;
  }
  console.table(
    [...rows]
      .filter(q => typeof q.ltime === 'number')
      .sort((a, b) => b.ltime - a.ltime)
      .slice(0, 10)
      .map((q, i) => ({
        rank: i + 1,
        ms: +(q.ltime * 1000).toFixed(3),
        component: q.trace?.component?.name ?? 'unknown',
        sql: q.sql
      }))
  );
})();
```

نکته نسخه‌ای: `QueryMonitorData` در Query Monitor 4 برای مرورگر آشکار است، اما ساختار تو در تو مثل `.data.db_queries.data.rows` یک جزئیات پیاده‌سازی است و ممکن است تغییر کند. اول از `Object.keys(...)` ساختار فعلی را بررسی کن. مقدار `ltime` در نمونه نصب‌شده Vinama ثانیه است؛ با ضرب ۱۰۰۰ به ms تبدیل می‌شود. **هیچ وقت SQL کامل دارای token یا اطلاعات حساس را در محل عمومی کپی نکن.**

۱۲.۲ گزارش محدود به SELECTها

```
const r = QueryMonitorData?.data?.db_queries?.data?.rows ?? [];
console.table(r.filter(q => /^\s*SELECT\b/i.test(q.sql || ''))
  .sort((a, b) => b.ltime - a.ltime)
  .slice(0, 10).map(q => ({ms: +(q.ltime * 1000).toFixed(3), sql: q.sql})));
```

۱۲.۳ فیلتر Non-SELECT

```
const r = QueryMonitorData?.data?.db_queries?.data?.rows ?? [];
console.table(r.filter(q => !/^\s*SELECT\b/i.test(q.sql || ''))
  .map(q => ({ms: +(q.ltime * 1000).toFixed(3), sql: q.sql})));
```

۱۲.۴ بررسی دستی Query Monitor Data

```
Object.keys(QueryMonitorData);
Object.keys(QueryMonitorData.data);
Object.keys(QueryMonitorData.data.db_queries.data);
console.log(QueryMonitorData.data.db_queries.data.rows[0]);
```

نکته: خروجی کل شیء ممکن است شامل URL یا مقادیر سایت باشد؛ قبل از اشتراک‌گذاری sanitize کن.

فصل ۱۳ — هفت مسیر سریع عیب‌یابی

مسیر A: صفحه کند است

1. ☐
همان URL با همان نقش و وضعیت کش بازتولید شد.
2. ☐
Overview: زمان کل، SQL و Peak Memory ثبت شد.
3. ☐
Timeline برای انتظار یا رویداد طولانی بررسی شد.
4. ☐
اگر SQL سهم قابل‌توجه دارد → DB Query → Component → Caller → Stack Trace.
5. ☐
اگر HTTP طولانی است → HTTP API → endpoint/timeout/caller.
6. ☐
اگر توضیحی پیدا نشد → PHP profiler / server logs / browser DevTools.
7. ☐
یک تغییر انجام و قبل/بعد چندبار مقایسه شد.

مسیر B: تعداد کوئری زیاد است

1. ☐
نوع درخواست و شمار کوئری‌ها ثبت شد.
2. ☐
Duplicate Queries و Callerهای پرشمار بررسی شد.
3. ☐
نمونه‌های N+1 از روی تغییر پارامتر و Stack Trace بررسی شد.
4. ☐
priming متادیتا، کش و حلقه‌های قالب بررسی شد.
5. ☐
تغییر، SQL Time را کاهش داد یا صرفاً تعداد را؟

مسیر C: خطا یا صفحه سفید PHP

1. ☐
خطا در محیط توسعه بازتولید شد.
2. ☐
PHP Errors و `debug.log`/ server error log بررسی شد.
3. ☐
فایل و خط ثبت شده و stack به کد افزونه/قالب دنبال شد.
4. ☐
نسخه PHP و رفتار deprecated API بررسی شد.
5. ☐
تغییر بدون فعال‌کردن نمایش عمومی خطا تأیید شد.

مسیر D: API کند یا خطادار است

1. ☐
HTTP API Calls مقصد و زمان را ثبت کرد.
2. ☐
status/timeout/DNS/TLS و call stack بررسی شد.
3. ☐

تکرار sync در مسیر page load بررسی شد.

4. ☐
caching، queue و retry policy منطقی آزموده شد.

مسیر E: قالب اشتباه یا 404

1. ☐
URL، query vars و نوع درخواست بررسی شد.

2. ☐
Template و template hierarchy بررسی شد.

3. ☐
Conditionals و rewrite rules بررسی شد.

4. ☐
بدون تغییر بی‌دلیل permalink، علت اصلاح شد.

مسیر F: کاربر دسترسی اشتباه دارد

1. ☐
با همان نقش واقعی بازتولید شد.

2. ☐
Capability Checks و stack فراخوانی مجوز بررسی شد.

3. ☐
role/capabilities و پلاگین مدیریت نقش مقایسه شدند.

4. ☐
در چند نقش و بدون افشای اطلاعات تست شد.

مسیر G: مشکل فقط گهگاه اتفاق می‌افتد

1. ☐
timestamp، URL، role و وضعیت cache ثبت شد.

2. ☐
تفاوت درخواست‌های خوب و بد مشخص شد.

3. ☐
Cron، HTTP خارجی، lockها و transientها بررسی شد.

4. ☐
برای شواهد ممتد از log/APM و مانیتورینگ سرور استفاده شد.

5. ☐
اصلاح در دوره زمانی کافی تأیید شد.

فصل ۱۴ — پرونده واقعی Vinama

محیط نمونه آموزشی: `D:\www\vinama` ، لوکال ویندوز، Query Monitor 4.x.

پرونده ۱: نبود Component

نشانه: هشدار قرمز: `unable to symlink its db.php file into place` ، نبود ستون Component و بخش Queries by Component.

شواهد: `wp-content/db.php` وجود نداشت.

اقدام: ساخت Symlink به فایل افزونه پس از بررسی وجود target و مجوز Windows.

نتیجه تأییدشده: هشدار حذف و Component/Rows و Queries by Component ظاهر شد.

درس: کیفیت تشخیص از خود ابزار شروع می‌شود؛ اگر Data Collector کامل نباشد، تحلیل انتساب ناقص است.

پرونده ۲: منوی فوتر

SQL: SELECT از `wp_posts` با JOIN به `wp_term_relationships` و `post_type = 'nav_menu_item'` .

Stack: `front-page.php → get_footer → footer.php → wp_nav_menu → wp_get_nav_menu_items → WP_Query` .

نتیجه: این کوئری برای دریافت آیتم‌های منو اجرا شده؛ خود وجود آن نشانه کندی نیست.

پرونده ۳: جدول گروه‌بندی SQL

در یکی از درخواست‌های سایت: **۹۵ کوئری** با مجموع **حدود ۵۳ms** SQL ثبت شد؛ بخش Core بیشترین سهم تجمعی را داشت. این مقدار **به‌تنهایی** دلیلی بر کند بودن هسته وردپرس نیست.

پرونده ۴: داده‌های QueryMonitorData

`QueryMonitorData.data.db_queries.data.rows` آرایه‌ای از آبجکت‌های دارای `sql` ، `ltime` ، `trace` و `result` بود. با مرتب‌سازی کپی آرایه، سه کوئری قابل‌مشاهده اول همگی UPDATE بودند؛ حدود **۲٫۱۶۳ms، ۳٫۸۰۵ms و ۱٫۸۳ms**. این زمان‌ها به‌تنهایی گلوگاه جدی نشان ندادند.
`component` همیشه به معنی «مالک داده» نیست: یک transient با نام مرتبط با WP Rocket ممکن است به Core نسبت داده شود؛ برای منشأ واقعی، trace لازم است.

فصل ۱۵ — فرم گزارش و چک‌لیست نهایی

فرم استاندارد یک پرونده عیب‌یابی

```
شناسه پرونده:
تاریخ / محیط:
نوع درخواست / نقش کاربر / URL:
وردپرس / PHP / وضعیت کش:
علامت مشکل و روش بازتولید:
Page Generation / Peak Memory / DB Time / Query Count:
HTTP API: مقصد / مدت / خطا:
SQL مشکوک (سانسورشده):
Caller / Component / Stack Trace:
EXPLAIN و شواهد تکمیلی:
فرضیه اصلی:
آزمایش کم‌خطر و نتیجه:
اصلاح انجام شده (commit):
Before / After، حداقل چند اجرای مقایسه‌ای:
نتیجه قطعی / موارد باز:
```

چک‌لیست قبل از نتیجه‌گیری

- ☐ تفاوت زمان SQL، زمان تولید پاسخ و زمان بارگذاری مرورگر را می‌دانم.
- ☐ تعداد کوئری را به‌تنهایی معیار کندی نگرفتم.
- ☐ Component را با فایل تعریف تابع یا مالک جدول اشتباه نگرفتم.
- ☐ از Stack Trace به نقطه فراخوانی مرتبط رسیدم.
- ☐ نتیجه EXPLAIN را با اندازه واقعی جدول و workload تفسیر کردم.
- ☐ روی سایت زنده اطلاعات محرمانه SQL/HTTP را افشا نکردم.
- ☐ تغییر قابل برگشت بود و با داده قبل/بعد تأیید شد.

فصل ۱۶ — واژه‌نامه و منابع

- **Caller:** تابعی که افزونه SQL را به آن نسبت داده است.
- **Component:** بخشی از وردپرس (هسته/افزونه/قالب) که افزونه عملیات را به آن منسوب کرده است.
- **Stack Trace:** زنجیره فراخوانی‌ها و مسیر اجرای کد.
- **Query Time:** زمان اجرای SQL مشخص.
- **Wall Time:** زمان واقعی سپری‌شده، شامل انتظارها.
- **Duplicate:** SQL تکراری؛ لزوماً N+1 نیست.
- **N+1:** بارگذاری مجموعه و اجرای درخواست مرتبط برای هر عضو.
- **Drop-in:** فایل خاصی مثل `wp-content/db.php` که وردپرس خیلی زود بارگذاری می‌کند.
- **Transient:** داده موقت با سیاست انقضا.
- **Autoload:** بارگذاری خودکار optionها هنگام راه‌اندازی WordPress.
- **EXPLAIN:** برنامه یا تخمین شیوه اجرای SQL از نظر دیتابیس.

منابع مرجع (بررسی‌شده در اکتبر ۲۰۲۶)

1. Query Monitor — https://querymonitor.com/
2. How to use Query Monitor — https://querymonitor.com/wordpress-debugging/how-to-use/
3. Database queries — https://querymonitor.com/wordpress-debugging/database-queries/
4. Query Monitor 4 — https://querymonitor.com/help/query-monitor-4/
5. Database drop-in / Symlink — https://querymonitor.com/help/db-php-symlink/
6. صفحه افزونه در وردپرس — https://wordpress.org/plugins/query-monitor/
7. MySQL EXPLAIN (مستندات Oracle) — https://dev.mysql.com/doc/refman/8.4/en/explain.html
8. WordPress Developer Resources — https://developer.wordpress.org/

پایان راهنما — نسخه ۱.۰. این کتابچه برای بازآزمایی و رجوع در عیب‌یابی طراحی شده است. هنگام انتشار نسخه جدید Query Monitor، رابط و ساختار `QueryMonitorData` را با مستندات رسمی تطبیق بده.